

Lecture 9:

Eigenvalues and Eigenvectors



Markus Hohle

University of California, Berkeley

**Numerical Methods for
Computational Science**

Course Map

Week 1: Introduction to Scientific Computing and Python Libraries

Week 2: Linear Algebra Fundamentals

Week 3: Vector Calculus

Week 4: Numerical Differentiation and Integration

Week 5: Solving Nonlinear Equations

Week 6: Probability Theory Basics

Week 7: Random Variables and Distributions

Week 8: Statistics for Data Science

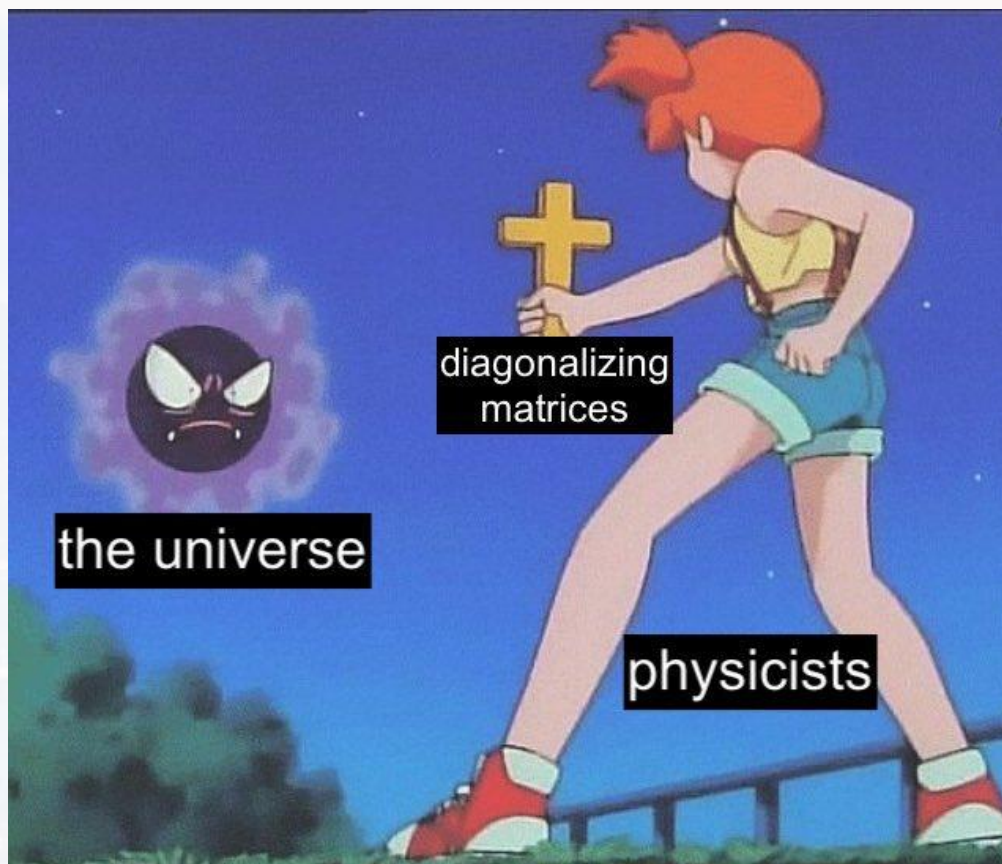
Week 9: Eigenvalues and Eigenvectors

Week 10: Simulation and Monte Carlo Method

Week 11: Data Fitting and Regression

Week 12: Optimization Techniques

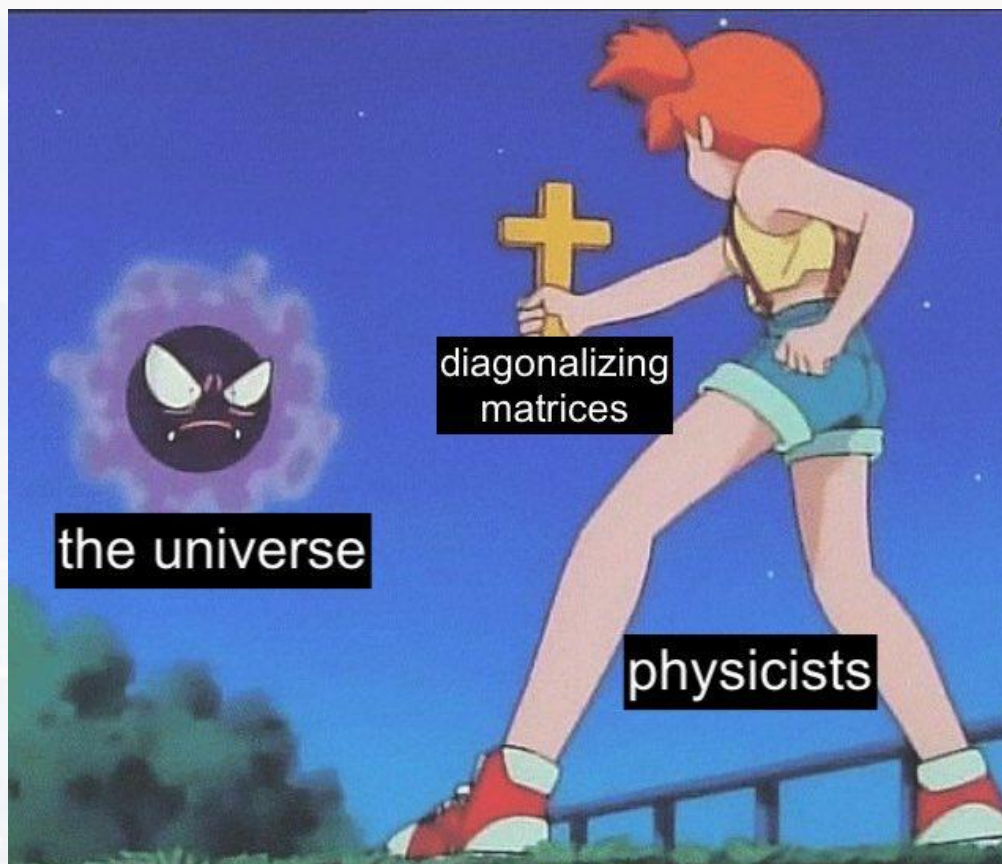
Week 13: Machine Learning Fundamentals



angryfermion

Outline

- A Geometrical Approach
- Finding Eigenvectors and Eigenvalues
- PCA
- Example I
- Example II



angryfermion

Outline

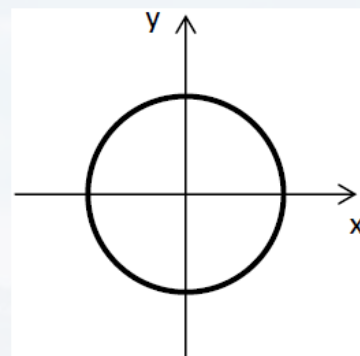
- A Geometrical Approach

- Finding Eigenvectors and Eigenvalues
- PCA
- Example I
- Example II



about quadratic forms

circle



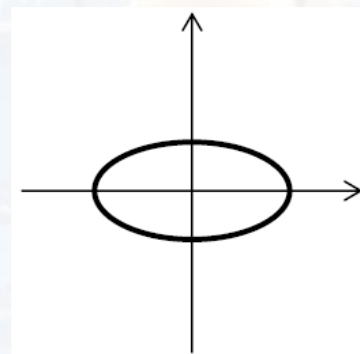
“distance to a reference point is constant”

$$x^2 + y^2 = \text{const} = r^2$$

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = \text{const}$$

$$a = b \rightarrow x^2 + y^2 = r^2$$

ellipse



“stretching the coordinate system”

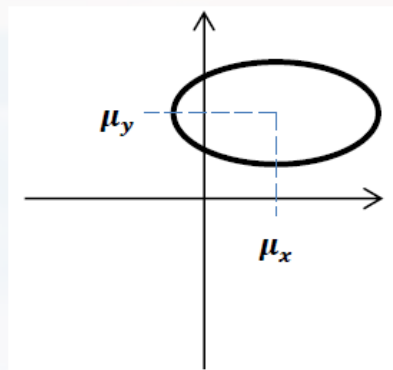
$$a \neq b$$

$$\frac{x^2}{a^2} + \frac{y^2}{b^2} = \text{const}$$



about quadratic forms

ellipse



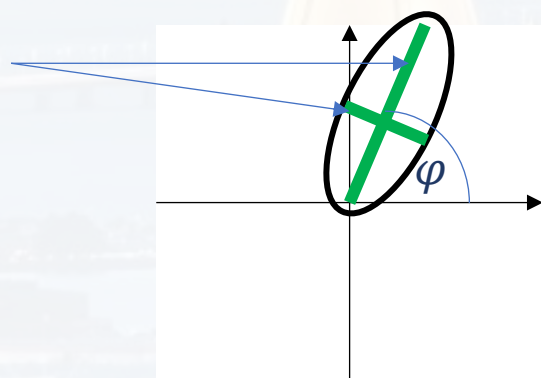
“moving the center of the ellipse”

$$\frac{(x - \mu_x)^2}{a^2} + \frac{(y - \mu_y)^2}{b^2} = \text{const}$$

$$x_0 = 0, x_0 \rightarrow \mu_x$$

$$y_0 = 0, y_0 \rightarrow \mu_y$$

principal
axes



“turning the ellipse by an angle φ ”

$$\varphi = \frac{1}{2} \operatorname{atan} \left(\frac{c}{\frac{1}{a^2} - \frac{1}{b^2}} \right)$$

$$\frac{(x - \mu_x)^2}{a^2} + \frac{(y - \mu_y)^2}{b^2} + c(x - \mu_x)(y - \mu_y) = \text{const}$$

turning the form



about quadratic forms

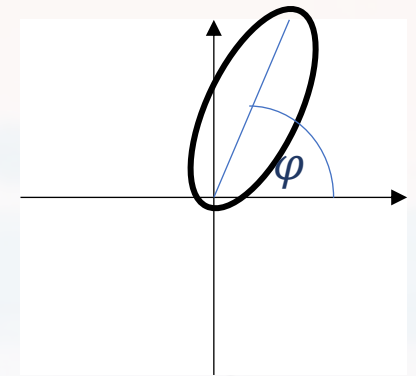
$$\frac{(x - \mu_x)^2}{a^2} + \frac{(y - \mu_y)^2}{b^2} + c(x - \mu_x)(y - \mu_y) = \text{const}$$

matrix form:

$$\text{const} = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} 1/a^2 & c/2 \\ c/2 & 1/b^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}$$

often:

$$\text{const} = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} A & C/2 \\ C/2 & B \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}$$



more general:

$$\text{const} = \begin{pmatrix} x - \mu_x \\ y - \mu_y \\ 1 \end{pmatrix}^T \begin{pmatrix} A & C/2 & D/2 \\ C/2 & B & E/2 \\ D/2 & E/2 & F \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \\ 1 \end{pmatrix}$$

depending on A, B, C, D, E, F

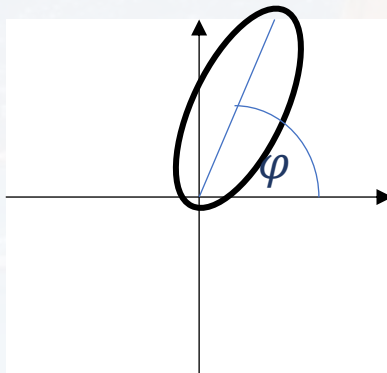
- circle
- ellipse
- parabola
- hyperbola



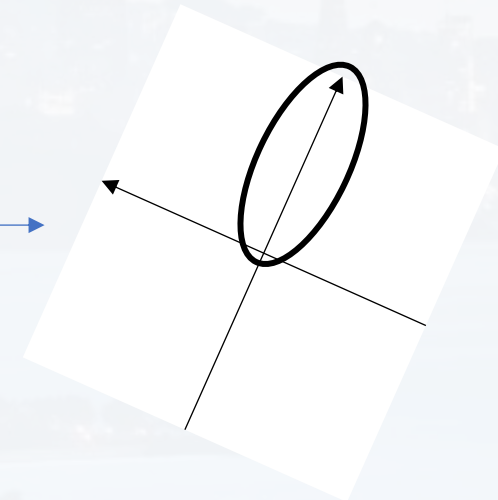
about quadratic forms

$$\frac{(x - \mu_x)^2}{a^2} + \frac{(y - \mu_y)^2}{b^2} + \boxed{c(x - \mu_x)(y - \mu_y)} = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} 1/a^2 & \boxed{c/2} \\ \boxed{c/2} & 1/b^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix} = \text{const}$$

turning the form



turning the coordinate system

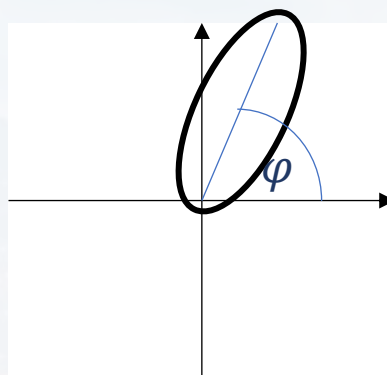


$$\frac{(x_{\text{new}} - \mu_{x(\text{new})})^2}{a_{\text{new}}^2} + \frac{(y_{\text{new}} - \mu_{y(\text{new})})^2}{b_{\text{new}}^2} = \begin{pmatrix} x_{\text{new}} - \mu_{x(\text{new})} \\ y_{\text{new}} - \mu_{y(\text{new})} \end{pmatrix}^T \begin{pmatrix} 1/a_{\text{new}}^2 & 0 \\ 0 & 1/b_{\text{new}}^2 \end{pmatrix} \begin{pmatrix} x_{\text{new}} - \mu_{x(\text{new})} \\ y_{\text{new}} - \mu_{y(\text{new})} \end{pmatrix} = \text{const}$$

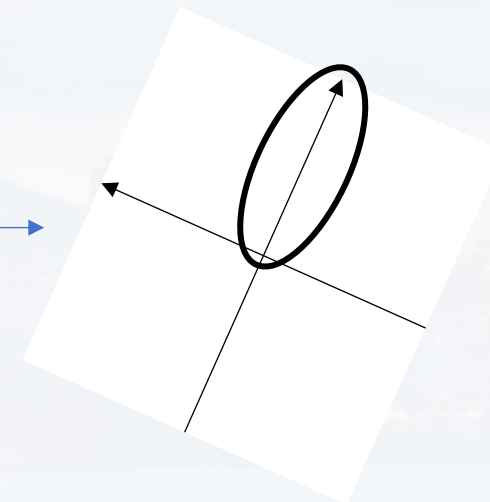


about quadratic forms

- non – diagonal elements: - turn/shear the object
- diagonal elements: - stretches (or flips, if negative) the object



turning the coordinate system



not turned/sheared

→ principal axes of the object are **parallel to the coordinate** axes

new coord. axis are called: **eigenvectors** $\vec{v}$ (“eigen”, German for “proper”)

→ they span the proper coordinate system!

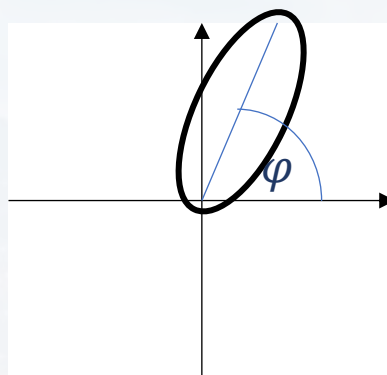
$$\begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_i & & 0 \\ 0 & 0 & & \lambda_N \end{pmatrix}$$

in the proper coordinate system: matrix is diagonal (entries are called **“eigenvalues”** λ)

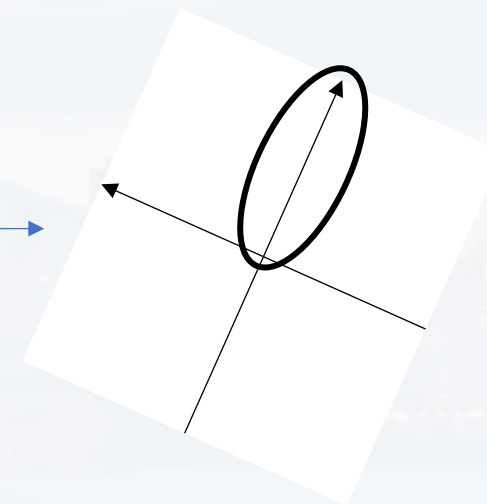


about quadratic forms

- non – diagonal elements: - turn/shear the object
diagonal elements: - stretches (or flips, if negative) the object



turning the coordinate system



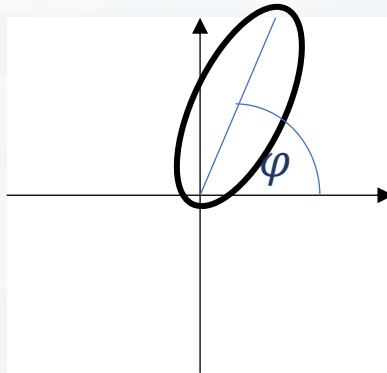
In a coordinate system in which the principal axes are **parallel to the coordinate** axes

- the matrix that defines the form is **diagonal**
- the **entries** of the now diagonal matrix are called **eigenvalues λ**
- the **axes** of this coordinates system are called **eigenvectors $\vec{v}$**
- eigen means “**proper**”, i. e. it is the “**most suitable**” coordinate system

$$\begin{pmatrix} \lambda_1 & 0 & \dots & 0 \\ 0 & \lambda_i & & 0 \\ 0 & 0 & & \lambda_N \end{pmatrix}$$

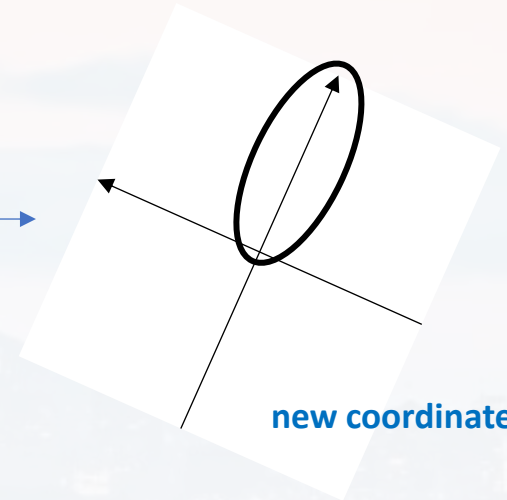


about quadratic forms



old coordinates

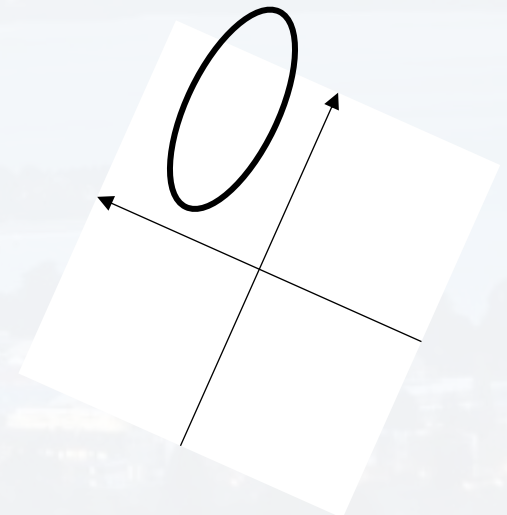
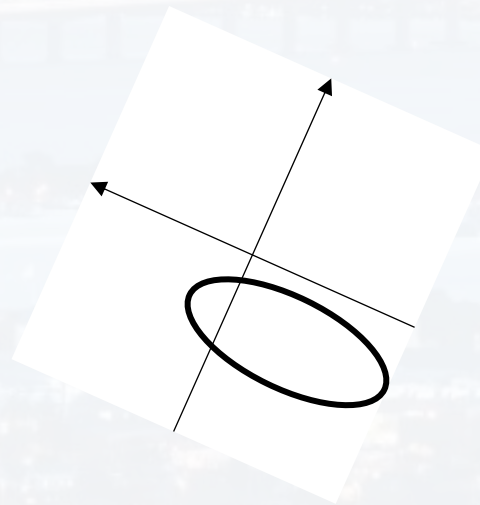
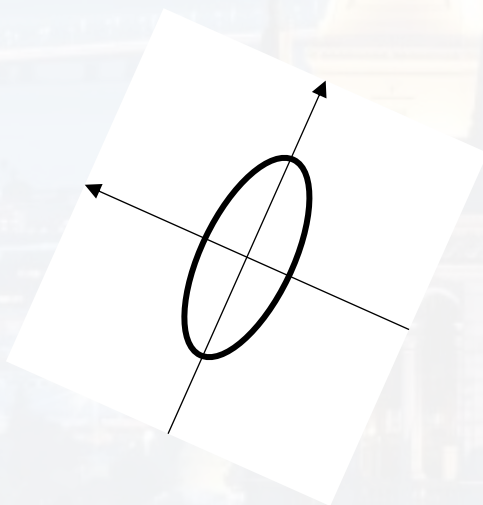
turning the coordinate system

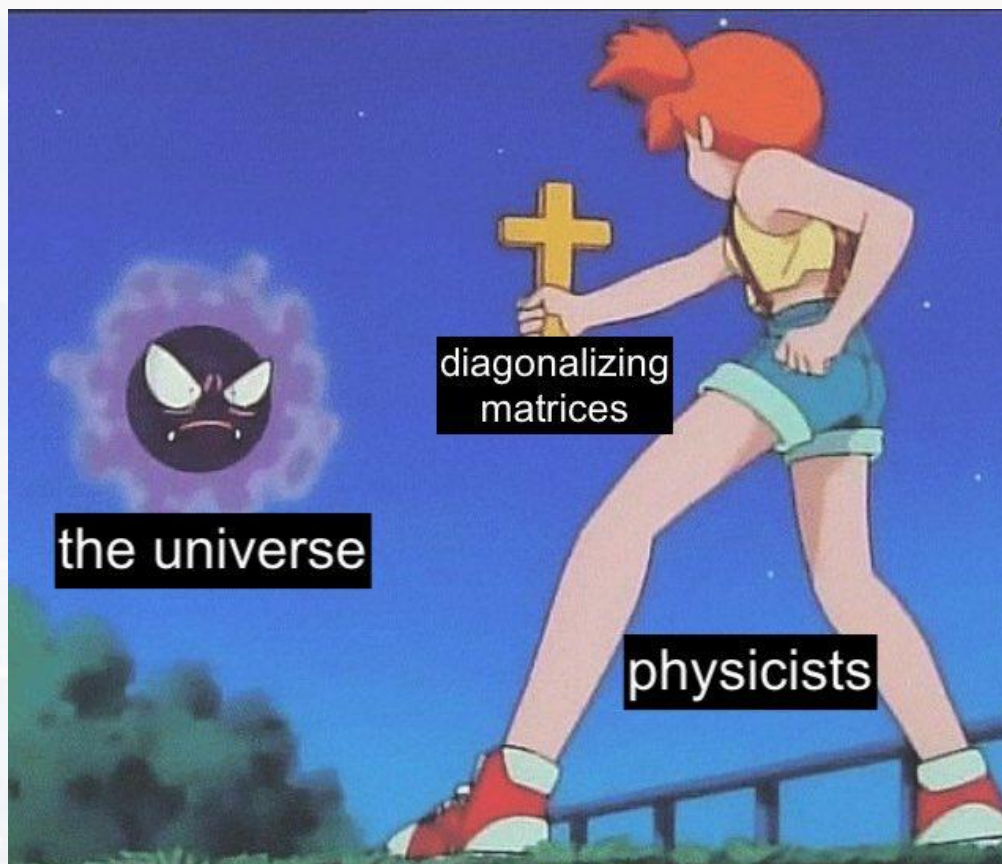


new coordinates

not turned/sheared

→ principal axes of the object are **parallel to the coordinate** axis





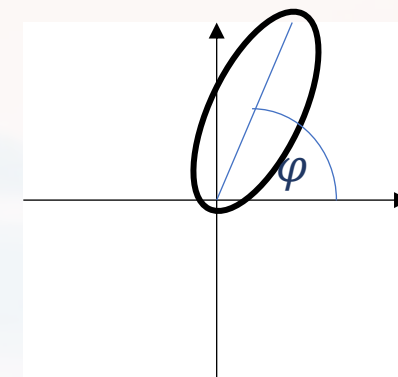
Outline

- A Geometrical Approach
- **Finding Eigenvectors and Eigenvalues**
- PCA
- Example I
- Example II



1) turned ellipse

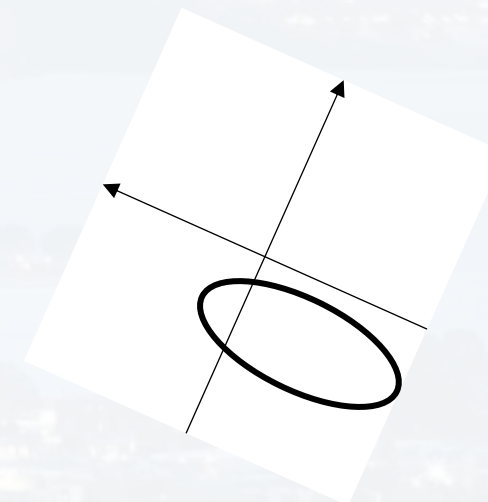
$$\begin{pmatrix} x \\ y \end{pmatrix}^T \underbrace{\begin{pmatrix} A & C/2 \\ C/2 & B \end{pmatrix}}_M \begin{pmatrix} x \\ y \end{pmatrix} = A x^2 + B y^2 + C xy$$



2) turning the coordinate system, such that principal axes of the are **parallel to the coordinate**

$$\begin{pmatrix} x_{new} \\ y_{new} \end{pmatrix}^T \underbrace{\begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}}_{M_{new}} \begin{pmatrix} x_{new} \\ y_{new} \end{pmatrix} = \lambda_1 x_{new}^2 + \lambda_2 y_{new}^2$$

eigenvalues λ



How to turn M into M_{new} ?



How to turn M into M_{new} ?

we assume, we have a set of eigenvectors $\vec{v}_i$

transforming M with $B = (\vec{v}_1, \vec{v}_2, \dots, \vec{v}_N)$ should turn M into a **diagonal matrix** M_{new}

$$M_{new} = B^T M B$$

after some algebra:

$$M \vec{v}_i = \lambda_i \vec{v}_i$$

which can be solved with:

$$\det(M - \lambda_i I) = 0$$

$$M \vec{v}_i = \lambda_i \vec{v}_i$$

characteristic equation

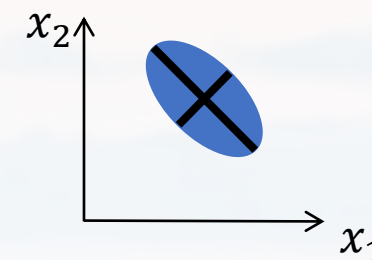


simple example:

$$\det(M - \lambda I) = 0$$

$$x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

$$M = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$



$$\det(M - \lambda_i I) = 0 = \det \left[\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} - \begin{pmatrix} \lambda_i & 0 \\ 0 & \lambda_i \end{pmatrix} \right] = \det \left[\begin{pmatrix} 2 - \lambda_i & -1 \\ -1 & 2 - \lambda_i \end{pmatrix} \right]$$

$$= 3 - 4\lambda_i + \lambda_i^2 = 0$$

characteristic polynomial

N eigenvalues and N eigenvectors
for N coordinates

$$\lambda_1 = 1 \quad \lambda_2 = 3$$



$$\lambda_1 = 1 \quad \lambda_2 = 3$$

$$\det(M - \lambda I) = 0$$

calculating the **eigenvectors** $\vec{v}_i$:

$$M\vec{v}_i = \lambda_i\vec{v}_i$$

characteristic equation

$$(M - \lambda_i I)\vec{v}_i = 0$$

$$\text{for } \lambda_1 \quad \left[\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} - \begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} \right] \begin{pmatrix} v_{1x} \\ v_{1y} \end{pmatrix} = \begin{pmatrix} v_{1x} & -v_{1y} \\ -v_{1x} & v_{1y} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad v_{1x} = v_{1y}$$

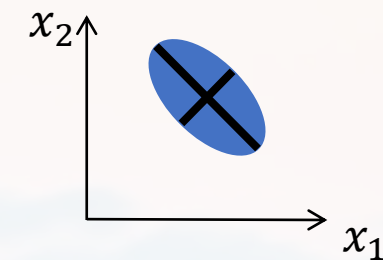
$$\text{e.g. } \vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

$$\text{for } \lambda_2 \quad \left[\begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} - \begin{pmatrix} 3 & 0 \\ 0 & 3 \end{pmatrix} \right] \begin{pmatrix} v_{2x} \\ v_{2y} \end{pmatrix} = \begin{pmatrix} -v_{2x} & -v_{2y} \\ -v_{2x} & -v_{2y} \end{pmatrix} = \begin{pmatrix} 0 \\ 0 \end{pmatrix} \quad v_{2x} = -v_{2y}$$

$$\text{e.g. } \vec{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$



$$x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad M = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \quad \boxed{\lambda_1 = 1 \quad \lambda_2 = 3}$$



$$\boxed{\vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}}$$

$$\boxed{\vec{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}}$$

recall: $B = (\vec{v}_1 \ \vec{v}_2)$ and $M_{new} = B^T M B$

$$M \text{ in the new coordinates is } \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix}^T \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix} \begin{pmatrix} 1 & 1 \\ 1 & -1 \end{pmatrix} = 2 \begin{pmatrix} 1 & 0 \\ 0 & 3 \end{pmatrix}$$

$\lambda_1 = 1$ (pointing to the top-left element 1)
 $\lambda_2 = 3$ (pointing to the bottom-right element 3)



$$x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix} \quad x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

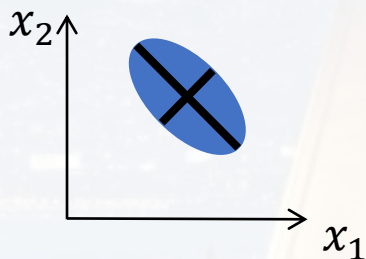
$$M = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

$$\lambda_1 = 1 \quad \lambda_2 = 3$$

$$\vec{v}_2 = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

$$\vec{v}_1 = \begin{pmatrix} 1 \\ 1 \end{pmatrix}$$

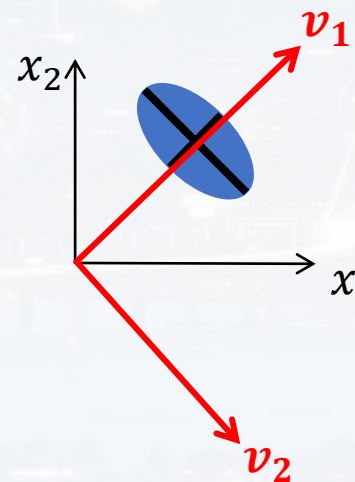
the old coordinates



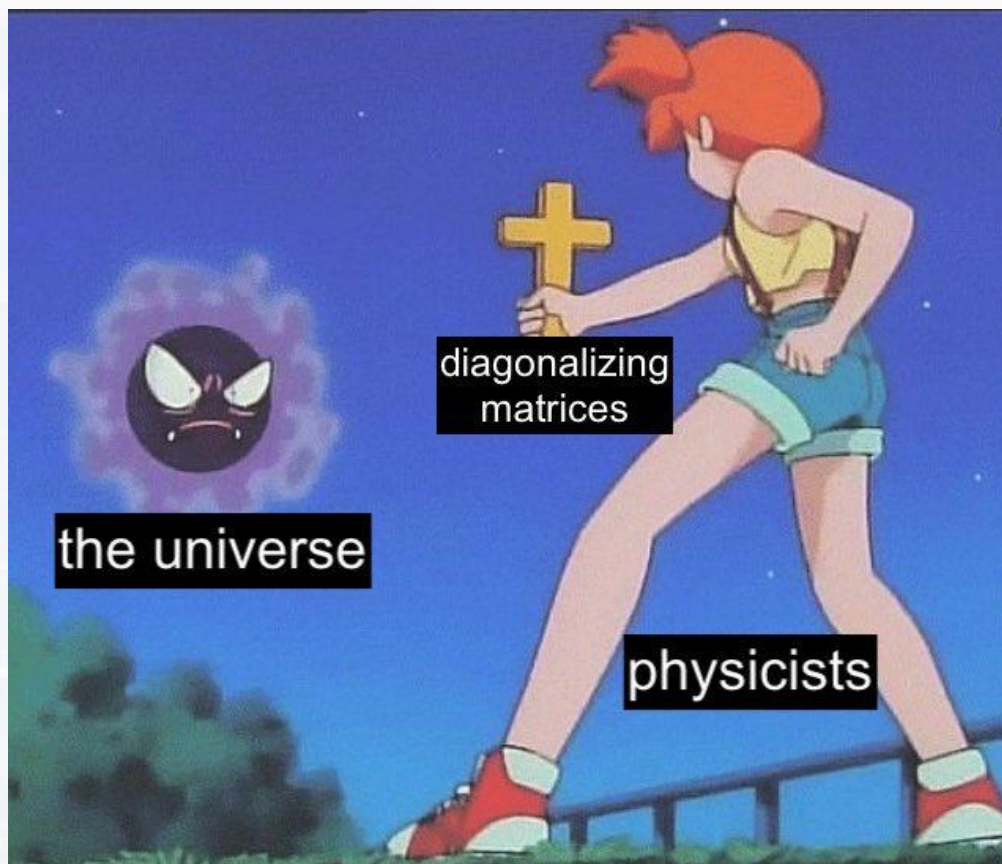
$$M = \begin{pmatrix} 2 & -1 \\ -1 & 2 \end{pmatrix}$$

$$x_2 = \begin{pmatrix} 0 \\ 1 \end{pmatrix} \quad x_1 = \begin{pmatrix} 1 \\ 0 \end{pmatrix}$$

the new coordinates



$$M_{new} = 2 \begin{pmatrix} 1 & 0 \\ 0 & 3 \end{pmatrix}$$



angryfermion

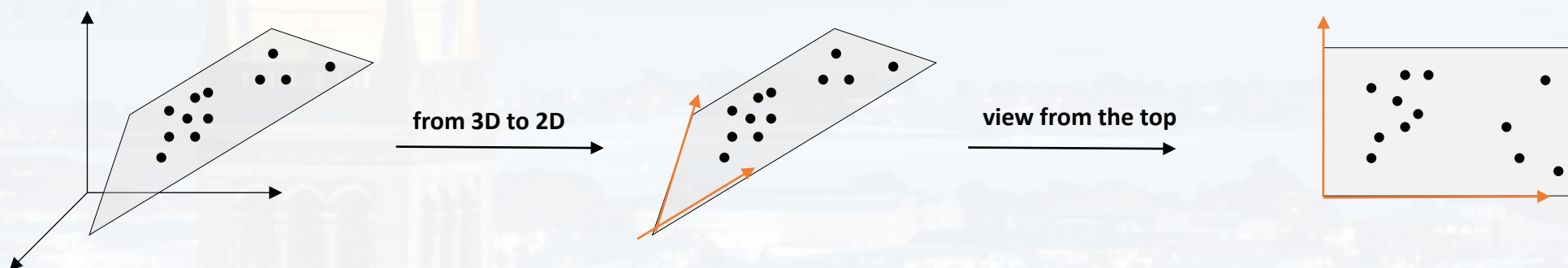
Outline

- A Geometrical Approach
- Finding Eigenvectors and Eigenvalues
- **PCA**
- Example I
- Example II



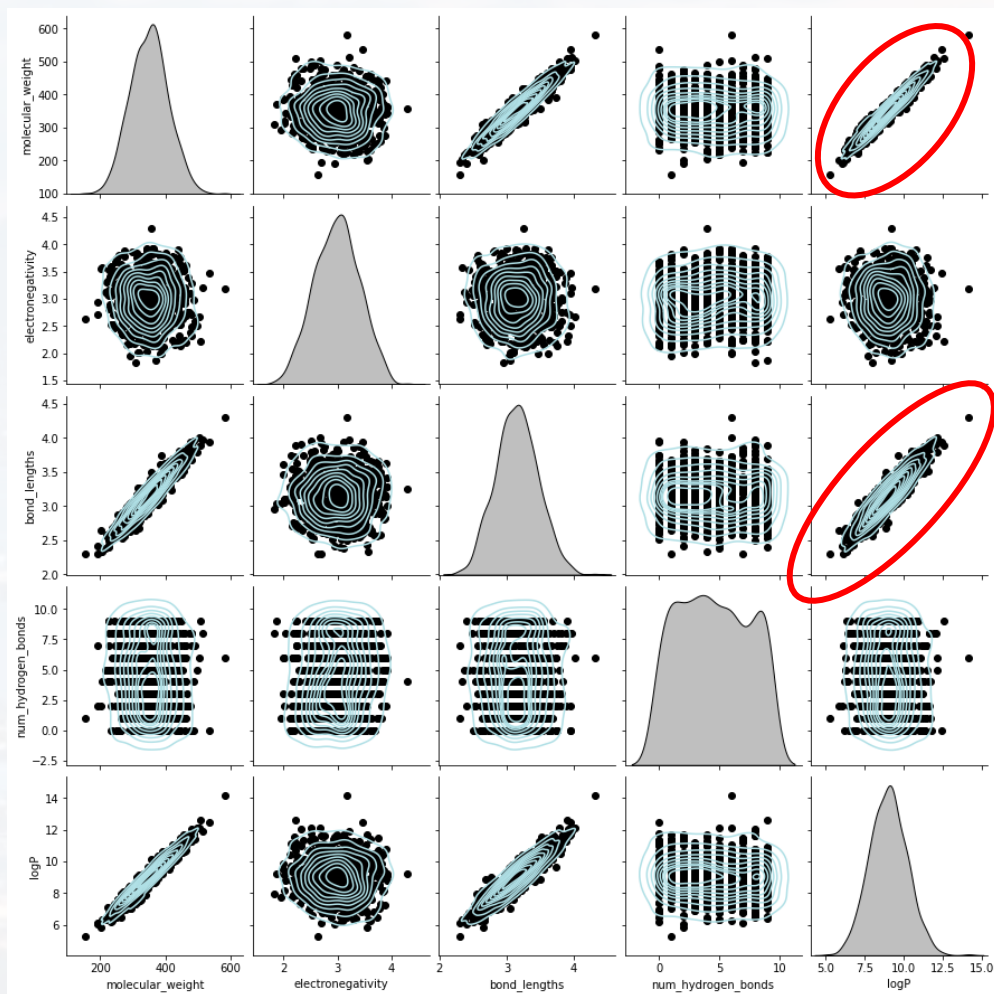
Goals:

- **Dimension reduction!**
 - Reducing the complexity of the dataset **without losing** information
 - Removing redundancies
 - Reducing the number of features
- trick: using **correlation** between different features





Lecture 8: correlation



	label	molecular_weight	electronegativity	bond_lengths	num_hydrogen_bonds	logP
	Toxic	382.602	2.00269	3.61153	3	9.82666
	Toxic	408.961	2.93626	3.47904	6	9.85889
	Non-Toxic	239.548	2.71413	2.63922	8	6.75962
	Non-Toxic	315.58	2.85598	2.86034	9	8.70674
	Non-Toxic	282.521	2.83877	2.9664	1	7.8173

$$\text{corr}(x, y) = \frac{\text{cov}(x, y)}{\sqrt{\text{var}(x)\text{var}(y)}}$$



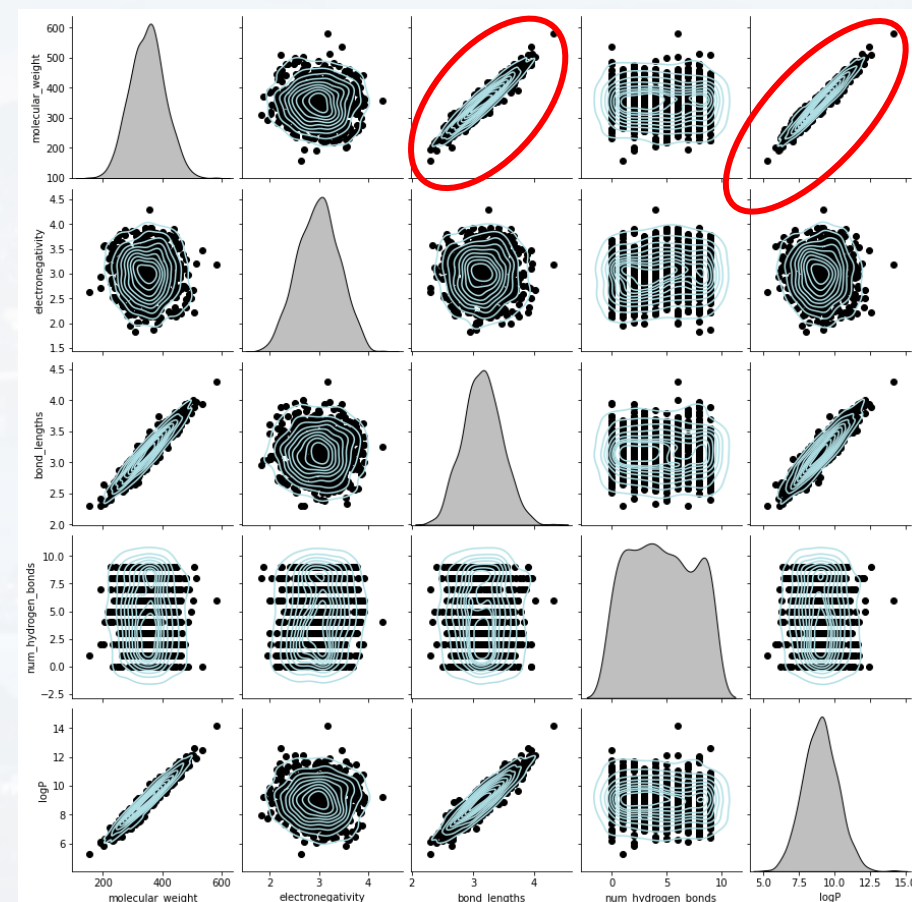


Lecture 8: correlation

correlation means:

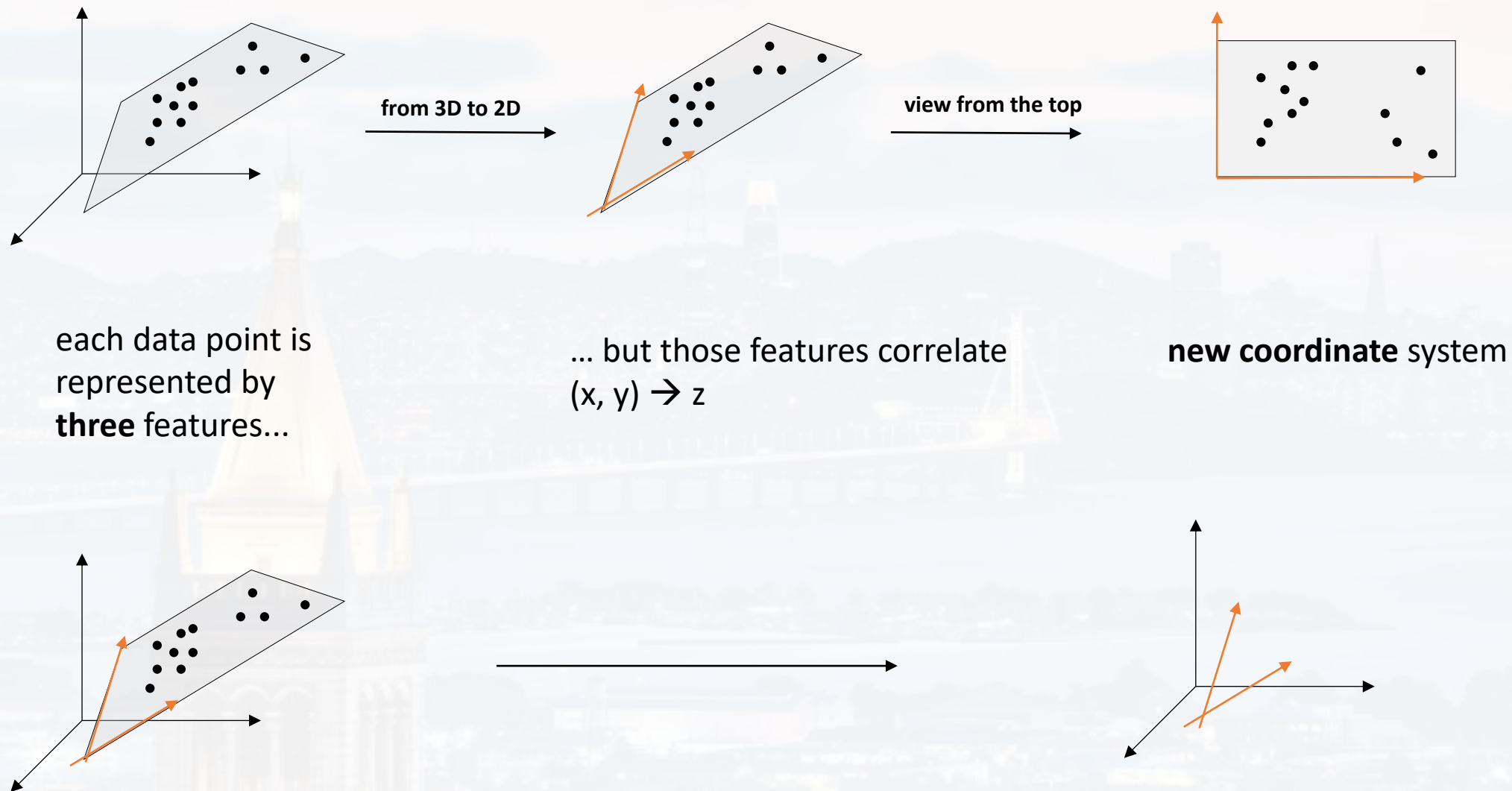
- features are **not mutually independent**
 - we can predict feature ***a*** from feature ***b*** to some extent
 - we don't need all features
- **reducing number of features** (dimensions) without losing information

label	molecular_weight	electronegativity	bond_lengths	num_hydrogen_bonds	logP
Toxic	382.602	2.00269	3.61153	3	9.82666
Toxic	408.961	2.93626	3.47904	6	9.85889
Non-Toxic	239.548	2.71413	2.63922	8	6.75962
Non-Toxic	315.58	2.85598	2.86034	9	8.70674
Non-Toxic	282.521	2.83877	2.9664	1	7.8173



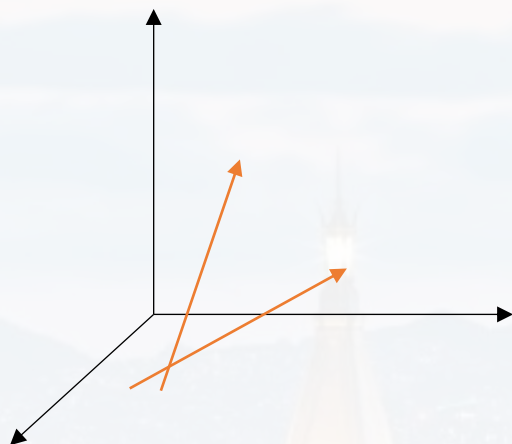


Lecture 8: correlation



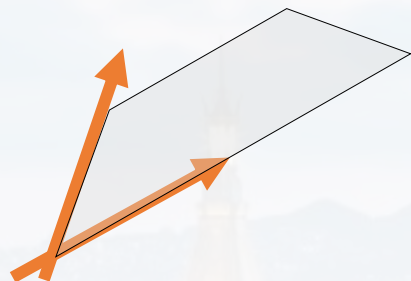


some features correlate!

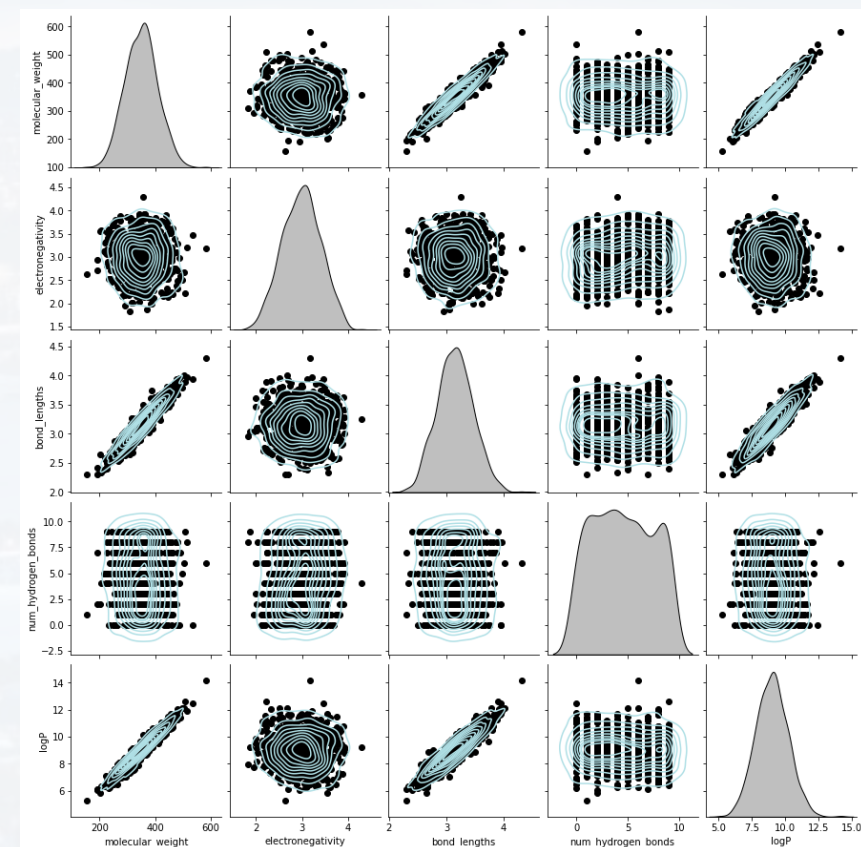




some features correlate!



How do we find these coordinates?





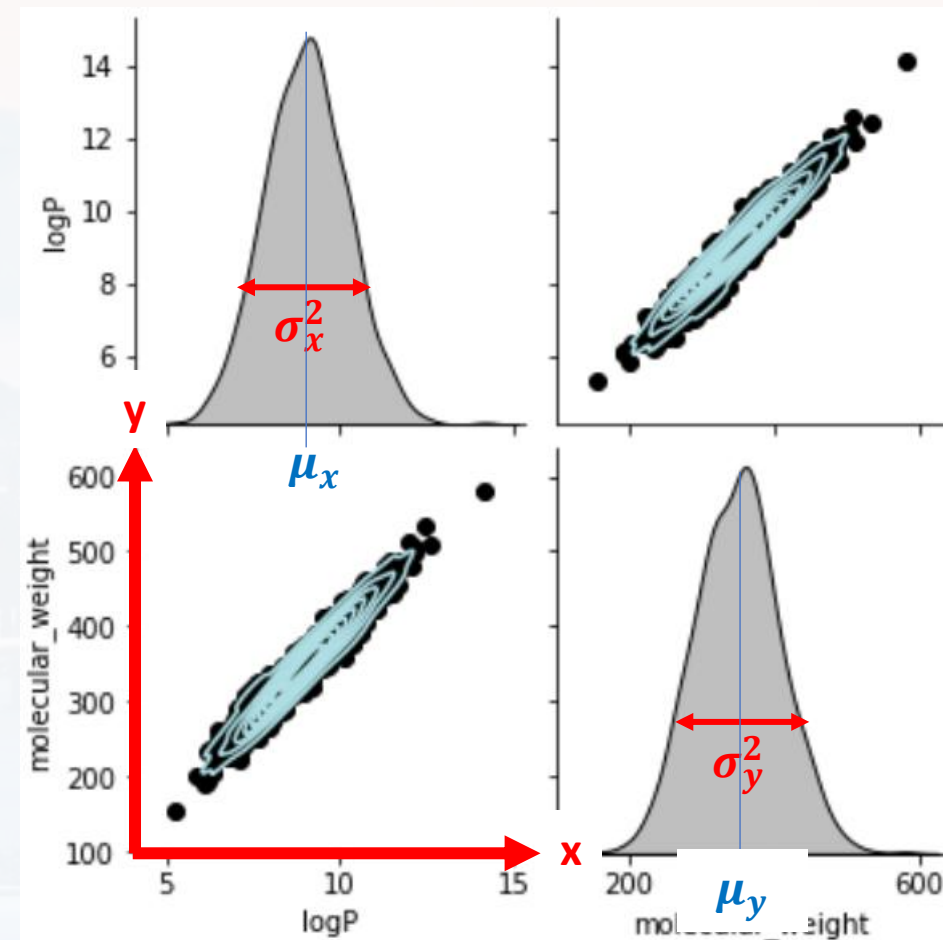
Lecture 8: correlation

$$\text{corr}(x, y) := \frac{\text{cov}(x, y)}{\sqrt{\text{var}(x)\text{var}(y)}}$$

$$\text{var}(x) \equiv \sigma_x^2 := \sum_i^N (x_i - \mu_x)^2$$

$$\text{cov}(x, y) := \sum_j^M \sum_i^N (x_i - \mu_x)(y_j - \mu_y)$$

$$\sigma_{tot}^2 = \boxed{\sigma_x^2} + \boxed{\sigma_y^2} + \boxed{2 \text{ cov}(x, y)}$$





$$\sigma_{tot}^2 = \boxed{\sigma_x^2} + \boxed{\sigma_y^2} + \boxed{2 \operatorname{cov}(x, y)}$$

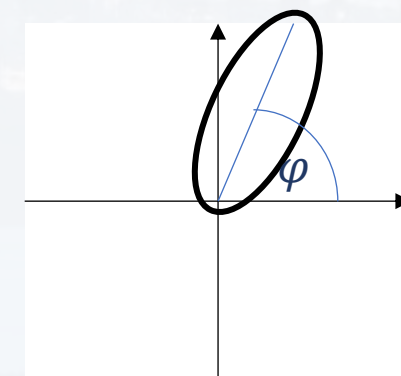
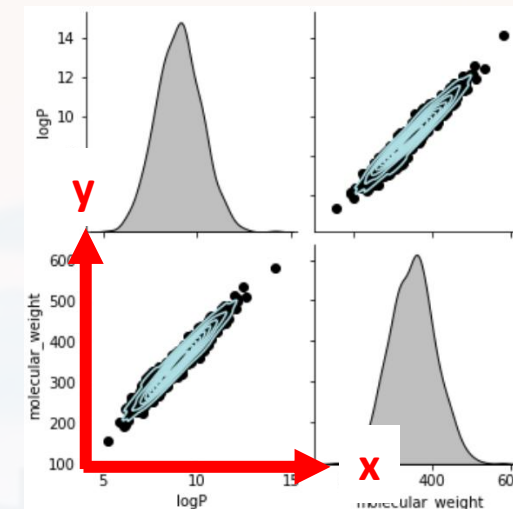
$$= \boxed{\sum_i^N (x_i - \mu_x)^2} + \boxed{\sum_j^M (y_j - \mu_y)^2} + \boxed{2 \sum_j^M \sum_i^N (x_i - \mu_x)(y_j - \mu_y)}$$

$$\text{const} = \boxed{\frac{(x - \mu_x)^2}{a^2}} + \boxed{\frac{(y - \mu_y)^2}{b^2}} + \boxed{2 c (x - \mu_x)(y - \mu_y)}$$

It is the same structure!

$$\text{const} = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} 1/a^2 & c \\ c & 1/b^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}$$

$$C = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} \sigma_x^2 & \operatorname{cov}(y, x) \\ \operatorname{cov}(x, y) & \sigma_y^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix} \quad \operatorname{cov}(y, x) = \operatorname{cov}(x, y)$$





$$\sigma_{tot}^2 = \sigma_x^2 + \sigma_y^2 + 2 \text{cov}(x, y)$$

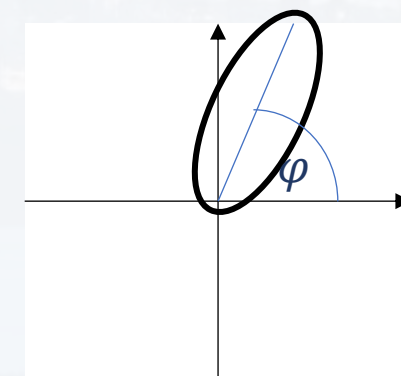
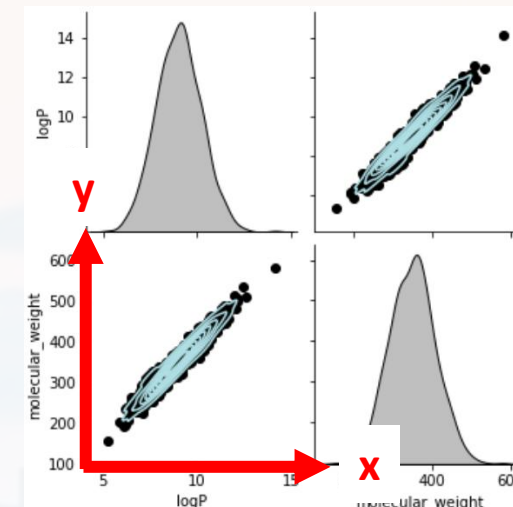
$$= \sum_i^N (x_i - \mu_x)^2 + \sum_j^M (y_j - \mu_y)^2 + 2 \sum_j^M \sum_i^N (x_i - \mu_x)(y_j - \mu_y)$$

$$const = \frac{(x - \mu_x)^2}{a^2} + \frac{(y - \mu_y)^2}{b^2} + 2c(x - \mu_x)(y - \mu_y)$$

It is the same structure!

$$const = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} 1/a^2 & c \\ c & 1/b^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}$$

$$C = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \begin{pmatrix} \sigma_x^2 & \text{cov}(y, x) \\ \text{cov}(x, y) & \sigma_y^2 \end{pmatrix} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix} \quad \text{cov}(y, x) = \text{cov}(x, y)$$





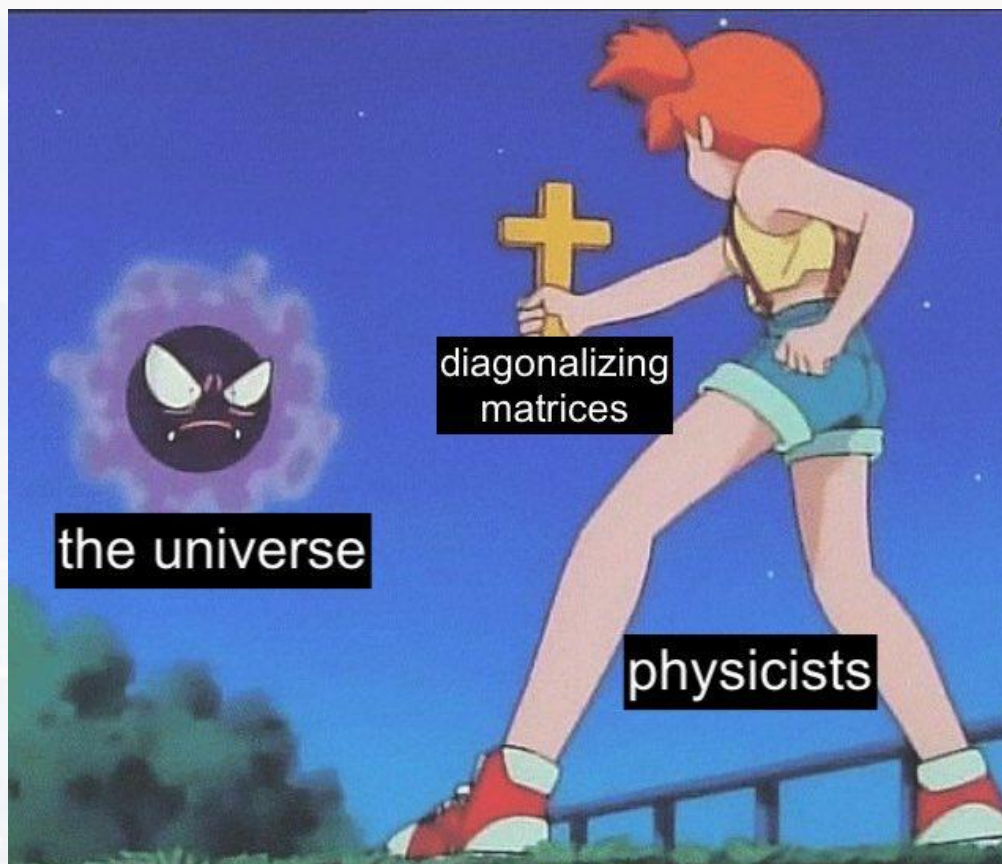
$$C = \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix}^T \underbrace{\begin{pmatrix} \sigma_x^2 & \text{cov}(y, x) \\ \text{cov}(x, y) & \sigma_y^2 \end{pmatrix}}_{\text{covariance matrix } \Sigma} \begin{pmatrix} x - \mu_x \\ y - \mu_y \end{pmatrix} \quad \text{cov}(y, x) = \text{cov}(x, y)$$

- geometrically, the **covariance matrix** can be interpreted as quadratic form
- the covariances are the **non-diagonal** elements of the **covariance matrix**
- aim: finding a coordinate transformation, where the **covariance matrix** is diagonal

$$\begin{pmatrix} \lambda_1 & \dots & 0 & \dots & 0 \\ 0 & & \lambda_i & \dots & 0 \\ 0 & & 0 & & \lambda_N \end{pmatrix}$$

the diagonal
entries are called
eigenvalues (= variances in
new coordinate system)

- all variables are **independent**
- principal components of the **covariance matrix**
are **parallel** to the **new coordinate axes** (= **eigenvectors**)



angryfermion

Outline

- A Geometrical Approach
- Finding Eigenvectors and Eigenvalues
- PCA
- **Example I**
- Example II

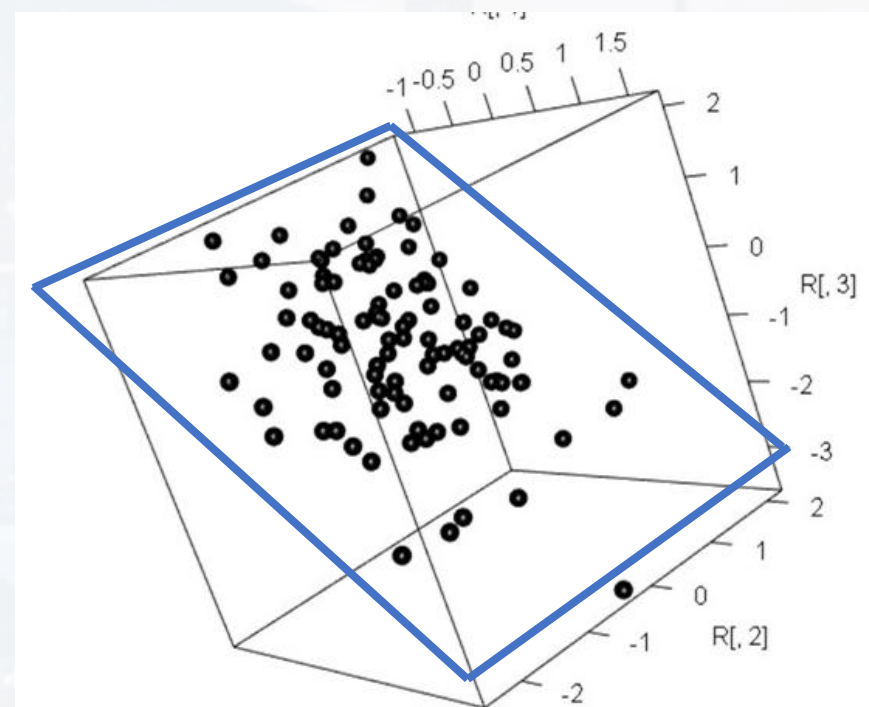


```
from sklearn.decomposition import PCA
```

Let us take a look at some artificial data first:

see **PCA_simple.ipynb**

- 3D data cloud
- however, all data points seem to be located on **one plane**
- PCA should be able to **reduce dimensions**





```
from sklearn.decomposition import PCA
```

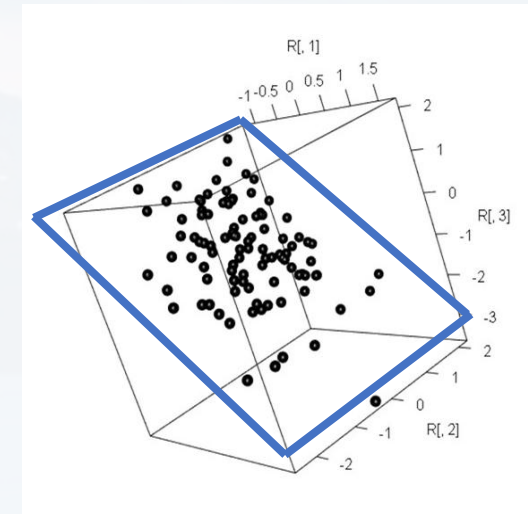
Let us take a look at some artificial data first:

```
XYZ = pd.read_csv('Rot.txt', delim_whitespace = True,\n                  header = None)
```

```
XYZ = np.array(XYZ)
```

```
fig = plt.figure(figsize = (12, 12))  
ax = fig.add_subplot(projection = '3d')  
ax.scatter(XYZ[:,0], XYZ[:,1], XYZ[:,2], c = 'black',\n           marker = 'o', s = 40)  
ax.set_xlabel('X')  
ax.set_ylabel('Y')  
ax.set_zlabel('Z')  
ax.tick_params(axis = 'both', which = 'major', labelsize = 30)  
plt.show()
```

- 3D data cloud
- however, all data points seem to be located on **one plane**
- PCA should be able to **reduce dimensions**





```
from sklearn.decomposition import PCA
```

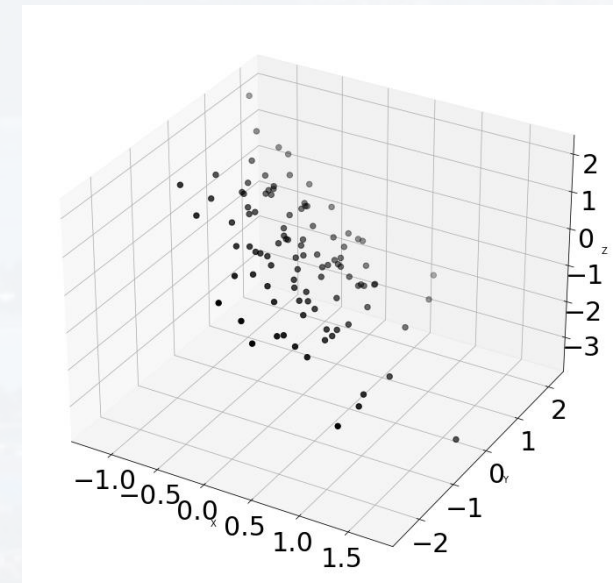
Let us take a look at some artificial data first:

```
XYZ = pd.read_csv('Rot.txt', delim_whitespace = True,\n                  header = None)
```

```
XYZ = np.array(XYZ)
```

```
fig = plt.figure(figsize = (12, 12))  
ax = fig.add_subplot(projection = '3d')  
ax.scatter(XYZ[:,0], XYZ[:,1], XYZ[:,2], c = 'black',\  
           marker = 'o', s = 40)  
ax.set_xlabel('X')  
ax.set_ylabel('Y')  
ax.set_zlabel('Z')  
ax.tick_params(axis = 'both', which = 'major', labelsize = 30)  
plt.show()
```

- 3D data cloud
- however, all data points seem to be located on **one plane**
- PCA should be able to **reduce dimensions**





performing the actual PCA:

```
out = PCA(n_components = 3).fit(XYZ)
```

```
eigenVec = out.components_  
eigenVal = out.explained_variance_  
eigenXYZ = out.transform(XYZ)
```

plotting the eigenvalue spectrum:

```
xplot = np.arange(1,4)
```

```
plt.bar(xplot, eigenVal, color = (0.8, 0.8, 0.8), edgecolor = 'black')  
plt.xlabel('dimension')  
plt.ylabel('eigenvalue')  
plt.yscale('log')  
plt.xticks(xplot)  
plt.show()
```

- 3D data cloud
- however, all data points seem to be located on **one plane**
- PCA should be able to **reduce dimensions**



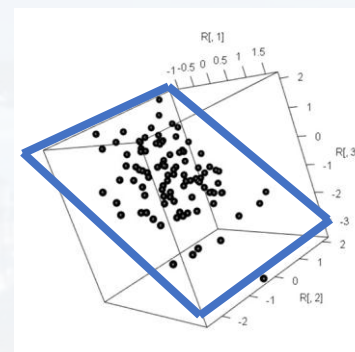
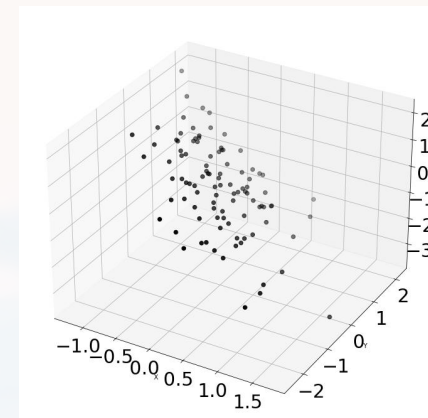
```
out = PCA(n_components = 3).fit(XYZ)
```

```
eigenVec = out.components_  
eigenVal = out.explained_variance_  
eigenXYZ = out.transform(XYZ)
```

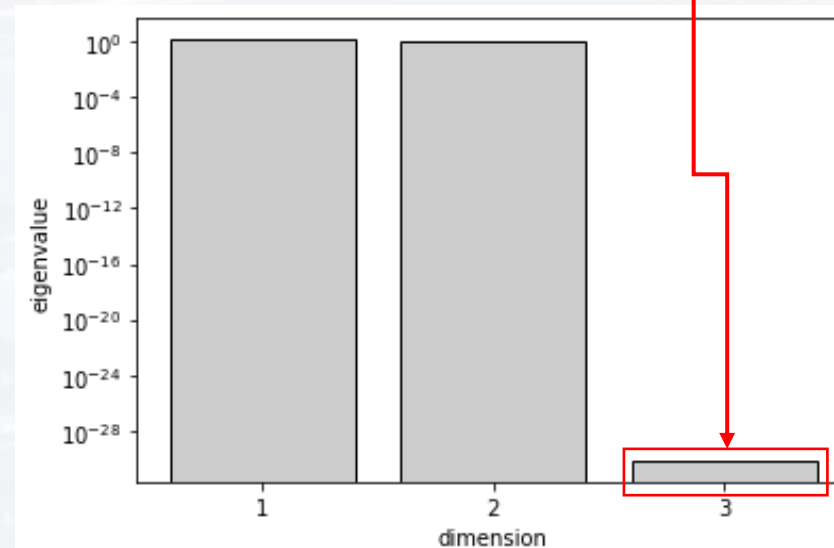
plotting the eigenvalue spectrum:

```
xplot = np.arange(1,4)
```

```
plt.bar(xplot, eigenVal, color = (0.8, 0.8, 0.8), edgecolor = 'black')  
plt.xlabel('dimension')  
plt.ylabel('eigenvalue')  
plt.yscale('log')  
plt.xticks(xplot)  
plt.show()
```



one eigenvalue
is zero

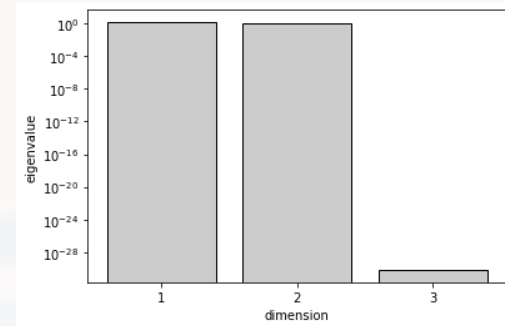




plotting the eigenvalue spectrum:

```
xplot = np.arange(1,4)
```

```
plt.bar(xplot, eigenVal, color = (0.8, 0.8, 0.8), edgecolor = 'black')  
plt.xlabel('dimension')  
plt.ylabel('eigenvalue')  
plt.yscale('log')  
plt.xticks(xplot)  
plt.show()
```



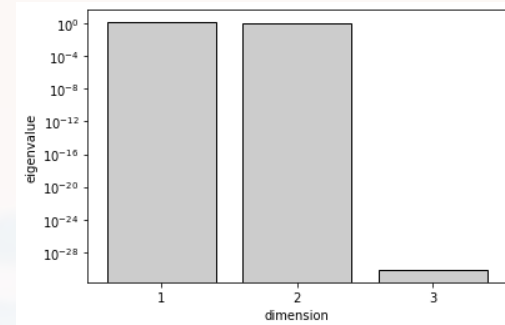
```
fig = plt.figure(figsize = (12, 12))  
ax = fig.add_subplot(projection = '3d')  
ax.scatter(eigenXYZ[:,0], eigenXYZ[:,1], eigenXYZ[:,2], c = 'black', \  
           marker = 'o', s = 40)  
ax.set_xlabel('X')  
ax.set_ylabel('Y')  
ax.set_zlabel('Z')  
ax.tick_params(axis = 'both', which = 'major', labelsize = 30)  
plt.show()
```



plotting the eigenvalue spectrum:

```
xplot = np.arange(1,4)
```

```
plt.bar(xplot, eigenVal, color = (0.8, 0.8, 0.8), edgecolor = 'black')  
plt.xlabel('dimension')  
plt.ylabel('eigenvalue')  
plt.yscale('log')  
plt.xticks(xplot)  
plt.show()
```

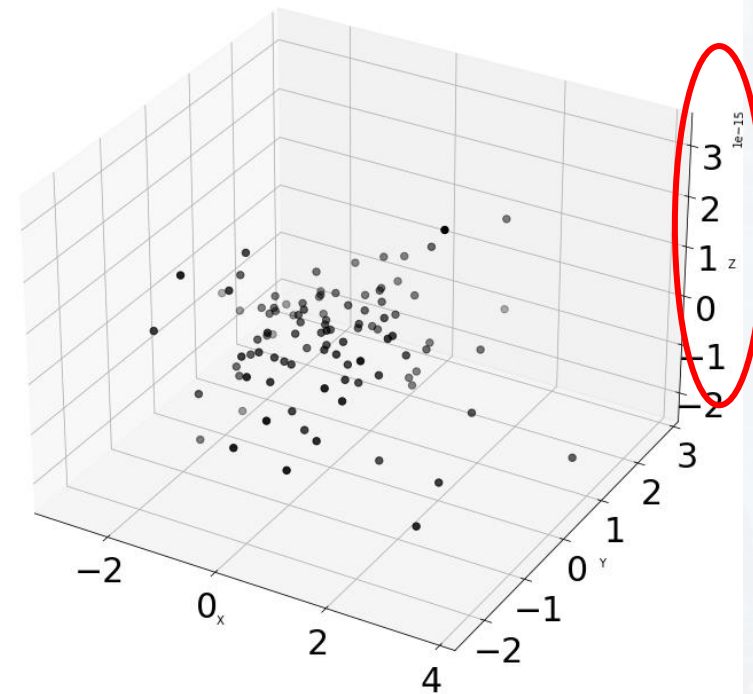


```
fig = plt.figure(figsize = (12, 12))  
ax = fig.add_subplot(projection = '3d')  
ax.scatter(eigenXYZ[:,0], eigenXYZ[:,1], eigenXYZ[:,2], c = 'bl')  
ax.set_xlabel('X')  
ax.set_ylabel('Y')  
ax.set_zlabel('Z')  
ax.tick_params(axis = 'both', which = 'major', labelsize = 30)  
plt.show()
```

check also eg:

```
np.dot(eigenVec[0,:],eigenVec[1,:])
```

almost no variance along new z-coord





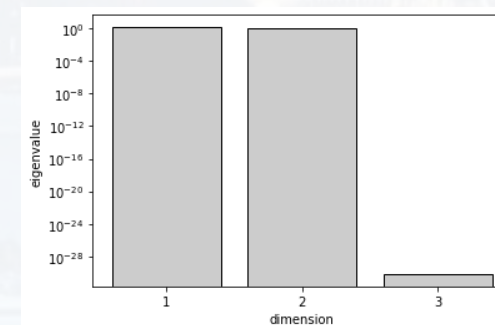
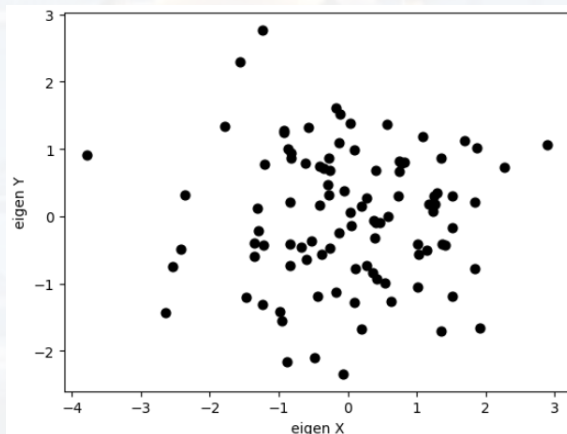
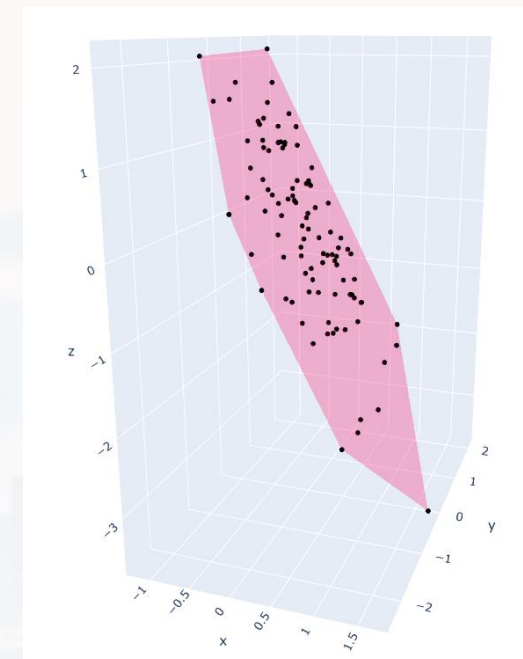
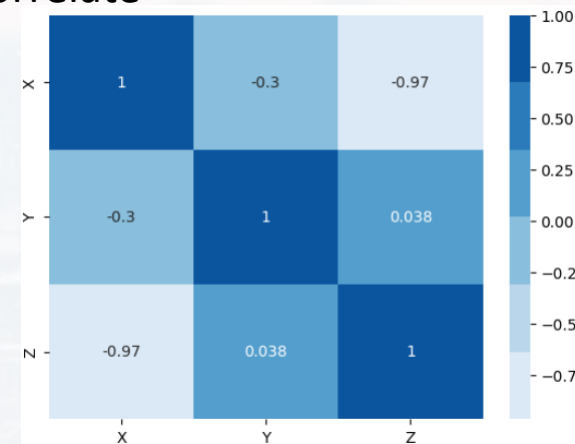
Summary:

- We don't need **three** coordinates in order to describe the data points
→ some of the directions (features) correlate

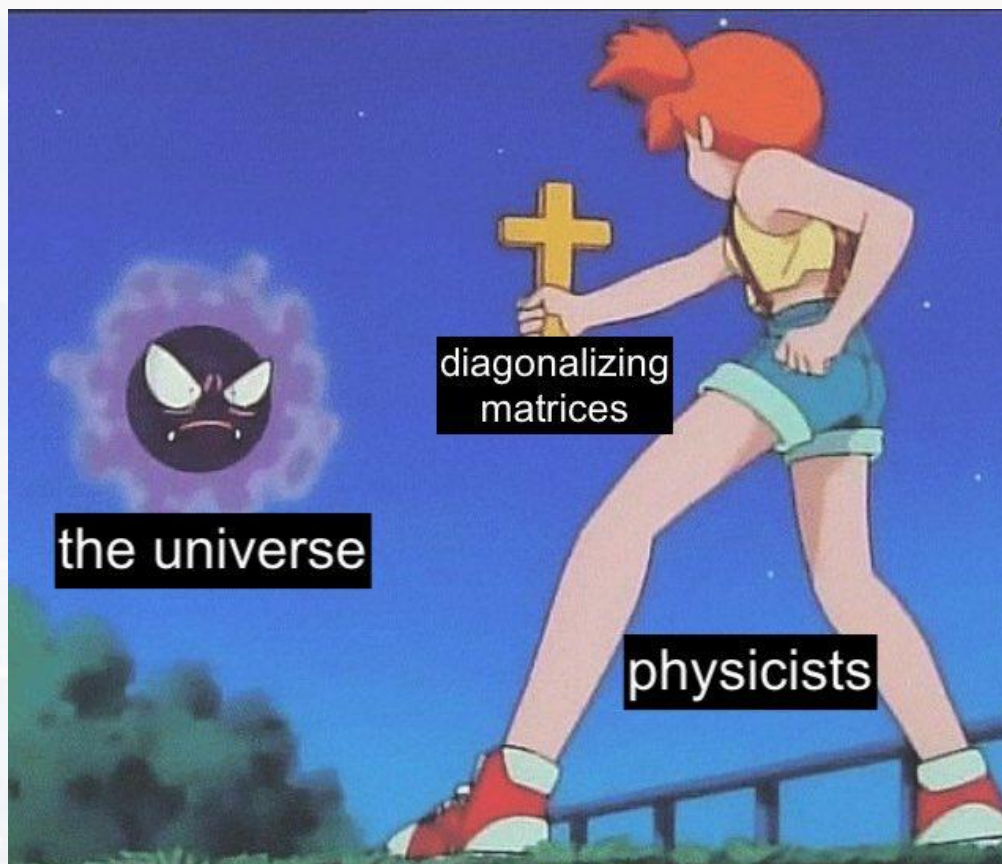
- running a PCA in order to find the proper coordinate system

- **one** of **three** eigenvalues is a lot smaller than the other **two**

→ We only need **two** coordinates for the data set



We can reduce the complexity of the data set without losing information



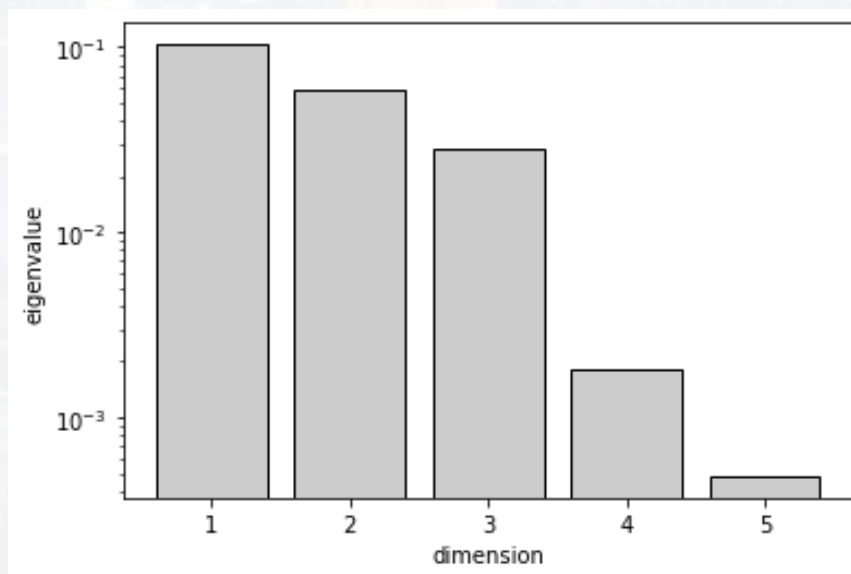
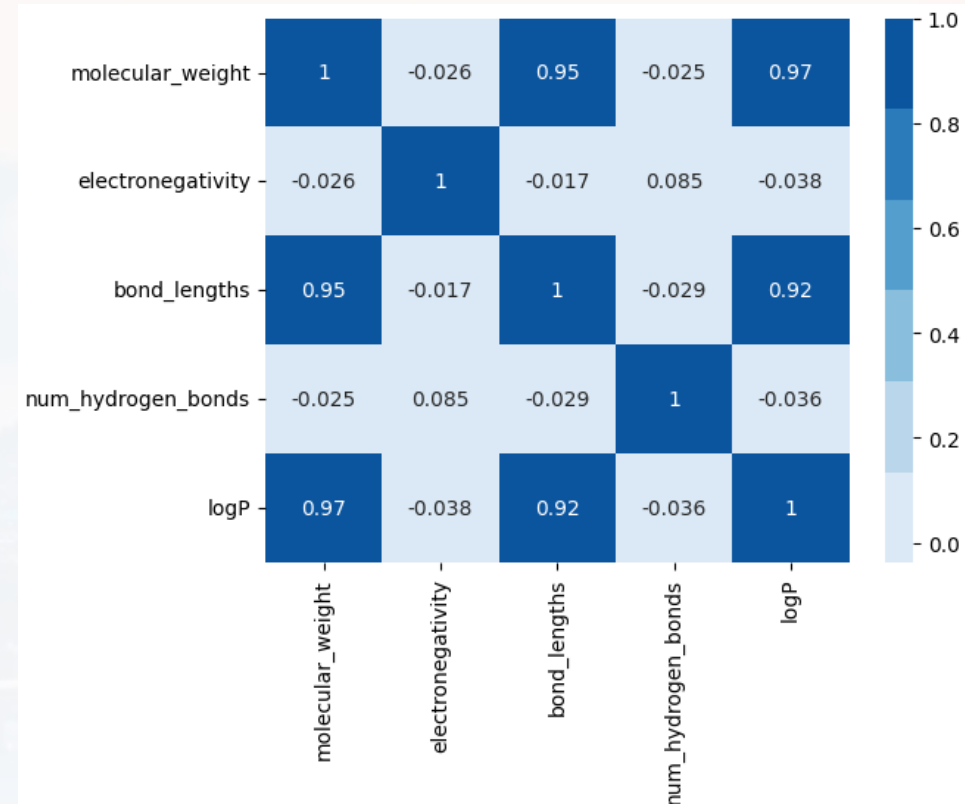
angryfermion

Outline

- A Geometrical Approach
- Finding Eigenvectors and Eigenvalues
- PCA
- Example I
- **Example II**



	label	molecular_weight	electronegativity	bond_lengths	num_hydrogen_bonds	logP
	Toxic	382.602	2.00269	3.61153	3	9.82666
	Toxic	408.961	2.93626	3.47904	6	9.85889
	Non-Toxic	239.548	2.71413	2.63922	8	6.75962
	Non-Toxic	315.58	2.85598	2.86034	9	8.70674
	Non-Toxic	282.521	2.83877	2.9664	1	7.8173



Lecture Exercise!



Thank you very much for your attention!

